# ELI — Grounding And Evidence

ELI v2.5.21

October 2026

## Contents


**ELI Grounding & Evidence Layer** **1**

The core idea . . . 1
Components . . . 2
Evidence-completeness gating (`contracts/grounded_control.py`) . . . 3
How it fits the pipeline . . . 4
Honest assessment . . . 4
ELI's account of its own behaviour (`cognition/self_claims.py`, 792 lines) . . . 4
Behaviours that keep answers grounded . . . 6
Deterministic reports are surfaced verbatim; examiner correctness . . . 7
News topic deepen (grounded, not guessed) . . . 7
Challenging a grounded answer must re-check it, not confabulate a new one . . . 8
History is not discarded before the budgeter sees it . . . 8
Retrieved evidence can't be cut away by an oversized-prompt truncation . . . 8


# ELI Grounding & Evidence Layer

> **Updated for v2.4.84.** Quick mode still returns verbatim for deterministic introspection; all CHAT modes now pass through the gradient orchestrator first.

The anti-confabulation system — a deterministic evidence scaffold wrapped around the probabilistic model. This is ELI's most distinctive subsystem and the part closest to genuinely frontier. Spans `eli/runtime/` and `eli/cognition/`.

## The core idea

For "grounded" actions (status, runtime, memory, identity, control), ELI does **not** trust the LLM to state facts. It (1) gathers deterministic evidence from the live system, (2) can answer directly from that evidence without the LLM at all, and (3) when the LLM does generate, validates the output against the evidence and rejects/repairs contradictions. The LLM becomes a phraser, not a source.

> **Correction (2026-06-08): the "bypass" is PARTIAL and MODE-GATED — not a blanket bypass.** Earlier wording here and in `what_eli_is.md` ("bypasses the language model entirely", "without the LLM at all") overstated it. The running reality, verified against the engine: the deterministic verbatim return fires for a **subset** of actions and is **mode-gated** — - a small **`_verbatim_always_actions`** set (deep introspection like `EXPLAIN_MEMORY_RUNTIME`) returns verbatim in **every** reasoning mode; - **`_deterministic_direct_payload_actions`** (status/reports + router-fast/command actions: DATE/TIME, VOLUME, WRITE_NOTE, window/media/file ops, NEWS/MORNING_REPORT, self-report …) return **verbatim in *quick* mode** and **synthesise in non-quick** modes (`engine._bypass_persona + _is_grounded_control_nonquick`); - everything else is the model's to phrase.

So the model IS in the loop for most conversational turns; the "phraser, not a source" principle holds where the deterministic path actually fires, not universally. A 2026-06-08 fix corrected the membership: the command actions were in the wrong set, so quick mode re-synthesised them, corrupting results ("Wrote note"→"Bought note") and adding latency. They are now verbatim in quick mode, synthesised in others.

## Components

### `runtime/deterministic_grounding_gate.py` (3.4k lines)

The deterministic renderer. `render_action(action, args, user_input, mode_label)` produces a grounded answer for control/status actions directly from runtime data (settings, runtime snapshot, DB counts, GPU line) — bypassing the model. `install(CognitiveEngine)` wires it into the engine. `_eli_v14_runtime_data()` assembles the live config block (model_path, n_ctx, gpu_layers, ...).

> **Structure.** `render_action` is one explicit ordered pipeline: each layer is a function that takes the layer below it as an argument, composed by `_stack(...)` into `_RENDER_PIPELINE`. An action no layer handles returns a JSON surface `missing_deterministic_renderer` rather than a made-up answer. The layer order is pinned by a characterisation test that compares the rendered output of every audited action.
>
> **`_eli_last_response_confidence_v2()`** (called for "how confident are you in your last answer") used to return a fixed template string regardless of what the last response actually was — a confident-sounding self-assessment that inspected nothing. It now reuses `eli.runtime.last_trace.load_last_trace() + control_contracts._trace_text()`, the same real per-turn trace mechanism already used correctly for "what was your last message," and says plainly when no trace is available rather than fabricating an answer.

### `runtime/control_contracts.py` (1,242 lines)

The deterministic control path: - `is_control_action` / `route_control_text` — recognise control/status intents. - `build_control_evidence(engine, action, args, …)` — gather the evidence packet (runtime paths, DB state, bus result, trace). - **`output_violates_evidence(text, evidence_text)`** — the gate that returns True when LLM output contradicts/omits the evidence; the engine uses this to reject a hallucinated answer. Seven concrete runtime terms (gpu layers, batch size, context size, cpu threads, model path, user/agent database) used to be blanket-exempted from this check, because evidence renders them under a machine-style key (`n_gpu_layers`, `model_path`...) while prose uses the human phrase — a plain substring match would reject a correct answer over spelling alone. That exemption also let the model state those exact values with *no* evidence backing at all. `_CONCRETE_TERM_ALIASES` now maps each term to every real spelling it appears under, closing the hole without reintroducing the false positives. - `compact_evidence_answer` / `finalise_control_result` — assemble the final grounded response.

#### Synthesis is validated against its evidence

`cognition/output_governor.validate_against_evidence` also rejects any figure or model designation (`H100`, `512 GB`, `32K`) that the evidence does not contain, allowing for unit conversion; the non-Quick compact synthesis path uses it and falls back to the deterministic evidence text. `contracts/grounded_control.py` owns which actions never fall back to a clarifying question.

### `runtime/evidence_ledger.py` (732 lines)

A persistent SQLite ledger of evidence events: `record_event`, `recent_events`, `actions_between` (what ELI executed in a period, per user, for the period log), `repeated_event_signals` (detect recurring issues over N days), `status_evidence`, `artifact_snapshot`. Gives ELI a durable, queryable record of what actually happened.

#### `runtime/evidence_arbitration.py` (212 lines)

`EvidenceItem` + `arbitrate_evidence(limit)` + `build_evidence_context_text` — scores and merges competing evidence into a single context block. Pairs with the agent-bus confidence aggregation (`_score_tool_result`), which scores a tool result with no recorded `"ok"` field as unverified (same low score as a confirmed failure) rather than defaulting a missing outcome to success.

#### Retrieval provenance

Memory hits reach the prompt through the shared turn retrieval in `memory/retrieval.py`. Each recalled line carries its event date and status (`memory_provenance.format_grounding_memory_line`) and the orchestrator adds a retrieval diagnostics block (what was searched, how many items, which time window) so ELI can say truthfully what its search did.

#### `cognition/output_governor.py` (1,694 lines)

Post-generation governor: `govern_output(text, is_grounded)`, `normalize_assistant_text`, `validate_against_evidence`, plus a family of drift-repair functions — `strip_generic_ai_identity_drift` (kills "As an AI language model…"), `repair_local_persona_drift`, `repair_self_user_confusion` (fixes the model conflating itself with the user), `clean_response_style`. The last line of defence before text reaches the user.

#### `cognition/grounded_status.py` (648 lines)

`direct_grounded_answer(user_text)` — fully deterministic answers for identity / memory-inventory / status questions, assembled from the DBs (`format_user_identity`, `format_memory_inventory`, table distributions). No LLM.

#### `runtime/grounded_remediation.py` (1.7k lines)

The failure→repair loop. When an action fails (app won't open, path missing, browser/IDE absent), it: `diagnose_app/path/browser/ide` → `build_repair_plan` → `offer_for_result` ("want me to install X?") → `handle_confirmation` (consumes yes/no) → `execute_pending_plan`. Stateful pending-repair tracking + `explain_last_failure`. This is what makes failures conversational and recoverable instead of dead ends. An install is offered only when a package source on the machine has the thing (`build_install_candidates` asks each one). When none has it the reply says so and, for a short name, asks whether to open its website instead: "open nasa" used to end "Would you like me to download/install it?", and a yes ran the same lookups again.

## Evidence-completeness gating (`contracts/grounded_control.py`)

A prior gap in this exact layer: `GROUNDED_CONTROL_ACTIONS` existed but was never checked before a direct/verbatim return or a no-clarify decision — only bare membership (`is_grounded_control_action`) gated anything, so a grounded action with genuinely incomplete evidence (a binary file decoded as text, an empty result) could still short-circuit straight to the user. `evidence_complete_for_action(action, local_vars)` closes that: it's checked at both live call sites — before the direct-payload verbatim return, and before the no-clarify-suppression decision — and on `False` the turn falls through to the next tier instead of returning early, rather than refusing outright.

- `GROUNDED_CONTROL_ACTIONS` (51 actions) is the union of `control_contracts.CONTROL_ACTIONS` and a new `FILE_EVIDENCE_ACTIONS` (10 actions: `SUMMARIZE_FILE`, `READ_FILE`, `ANALYZE_PDF[_FOLDER]`, `ANALYZE_IMAGE`, `OCR_IMAGE`, `TRANSCRIBE`, `CONVERT_DOCUMENT`, `FILE_AUDIT`, `SCREEN_READ_ANALYZE`) — the family implicated in a real bug this closes: a `.docx` read as raw binary and confabulated into a fake analysis, because nothing checked whether the "evidence" was actually decodable text before treating it as complete.

- `_looks_like_file_evidence_complete` runs a printable-ratio + non-empty + no-error-marker check on file-family evidence — this is what would have caught the binary-as-text bug directly. A minimum-length floor (20 chars) also treats a bare dict-key-name leak (`_collect_text` including Mapping keys, not just values) as still-incomplete rather than a false "complete."
- `evidence_gap_reason(action, local_vars)` returns `"complete"`, `"missing"` or `"corrupted"` — a typed reason, not just a bool, for callers that want to say *why* evidence fell short.
- Kill switch: `ELI_EVIDENCE_GATE_DISABLE=1` bypasses the check at both call sites, since widening `GROUNDED_CONTROL_ACTIONS` from the old hand-maintained ~10-action list to the full ~51-action union means the no-clarify gate now applies far more broadly than before — this lets a false-refusal in the field be killed instantly without a revert.
- The generic fallback (non-file, non-runtime-status actions) stays weak — satisfied by dict keys existing at all, not real content — an acknowledged, separate gap, not part of this pass.

`tests/test_grounded_control.py` covers it, including a direct repro of the original binary-garbage bug (asserts `evidence_complete_for_action` now returns `False` on it).

## How it fits the pipeline

1. Router classifies action. Control/grounded actions enter the deterministic path.
2. `build_control_evidence` gathers facts.
3. PHASE45 deterministic bypass: for many status actions `render_action` / `direct_grounded_answer` answer **without** the LLM.
4. If the LLM generates, `output_violates_evidence` + `validate_against_evidence` gate it; `govern_output` cleans drift.
5. `grounded_remediation` handles action failures with an offer/confirm loop.
6. `evidence_ledger` records the event for future `repeated_event_signals`.

## Honest assessment

- **Strong (genuinely):** very few local-LLM projects build a deterministic evidence layer at all, let alone one this thorough — direct-answer bypass, output-vs-evidence rejection, drift repair, a persistent ledger, and a conversational remediation loop. This is the subsystem that most justifies the "frontier" label.
- **Weak:**
  1. The gate file is still 3.4k lines with many version-named helpers (`_eli_v9_*`, `_eli_v14_*`); the layers are ordered explicitly now, but the helpers wait to be folded into fewer, named modules.
  2. **Overlapping surfaces** — `grounded_status`, `control_contracts`, `deterministic_grounding_gate`, and the `runtime/*_response` / `*_surface` modules all render grounded answers with partial overlap. The boundaries between "who renders the final grounded string" are blurry.
  3. Confidence/grounding is heuristic (additive score + threshold + `output_violates_evidence`), not a formal proof — fine, but it's a gate, not a guarantee.

---

## ELI's account of its own behaviour (`cognition/self_claims.py`, 792 lines)

Routing, retrieval and escalation run before the model is called and never reach it, so asked "why did you search the web?" a model can only invent. Live, a 35B model blamed "vector search drift", offered a `verbose_media_logging` setting and an `action_logs` table that do not exist, and said "the audit trail is now active" for logging that was always on.

- **The turn record.** Each turn notes what the pipeline did (`request_context.note_turn_fact`): the rule that routed it, a downgrade to chat, what retrieval found, whether it escalated to the web and why, every action that ran, how often the model ran, on how many prompt characters and for how many seconds. `describe_turn` writes that into the turn's signed audit row (`orchestrator_audit_ledger`), so the row answers "why did you do that" later.

- **Asked about itself, ELI is handed the record.** For a question or complaint about its own behaviour (`asks_about_own_behaviour`), the recent audit rows, each paired with what the user asked, go beside the recent exchange just before the message, where the overflow trim cannot cut them (in the middle of the system prompt they were cut). The same holds for an insult aimed at ELI, a correction ("I did not ask about X", "I asked why …"), and a bare reaction with no subject of its own ("What the fuck is going on!?", "seriously??", `reacts_to_last_reply`); none of these goes to the intent model. Direct questions about the last turn ("why did that take so long", "what did you just do", "why did you search the web"), and a remark that it was slow ("that took over a minute??"), are answered from those rows with no model call (`ROUTING_FAULT_EXPLAIN`); asked about time, the last turn comes first with where the model runs. The correction shortcut never replaces such an answer, or any other action that answers or acts directly. None of these turns is ever escalated to a web search.
- **A question about something already said is about the conversation.** "When was that, in that conversation?", "your long response above", "the response from you" (`correction_patterns.refers_to_earlier_re`) are never web-searched; the history block then gives every earlier reply of ELI's whole, and a correction that asks one (`that is not true, what day is that from?`) leaves the correction shortcut, which only sees the last exchange, for the full pipeline.
- **The recent exchange keeps what a follow-up is about.** It is built newest first within its share of the context window (`session_continuity.exchange_budget_chars`): ELI's last reply is whole, a fetched result (a news briefing, a search) stays whole for three replies and is labelled "result of NEWS_FETCH, not recalled", and older turns are clipped. When a prompt is still too long, the oldest lines go, never the message or the newest turns (`fit_exchange_prompt`). Before 2.5.15 every reply was cut to 280 characters and an oversized prompt was cut from the front: a follow-up on a story past the cut reached a model that had never seen it, and it called a real Guardian story its own invention.
- **A correction is answered, not re-run.** When a correction or complaint about the last reply routes to the very action that produced it, it goes to chat with the record and that reply whole ("I did not ask about RAM" ran the RAM report again, word for word).
- **The claim check** (`gate_stream` on streamed replies, `drop_invented_self_claims` in `govern_output`). A sentence is dropped when it reports a change nothing made ("logging is now active", when no action ran); promises a standing change a reply cannot make ("no web search from now on"); names a setting, table, module or file ELI does not have (checked against the real actions, settings, tunables, tables, modules and the filesystem); proposes a fix ELI has no action for; or, on a fault question, explains a cause the record does not carry (most of the sentence's content has to be written in the audit rows or the period log). A sentence that rests on a dropped one ("That is why…", "Want me to run that now?") goes with it, and a heading left with nothing under it is removed. ELI's own earlier replies are never counted as evidence. When sentences were dropped the reply says so and gives the recorded rows instead; code blocks are never touched.
- **A reply cannot say it did something when nothing ran.** On a turn in which no action was carried out, a sentence is dropped when it reports an act on the machine: "I've opened …", "Spotify is opening now", "Scanning `/path` …", "(Waiting for system execution output…)", "Project `X` initialized." The verbs are the ones a command starts with, less the work a reply does in its own words ("I've summarised it below" is the reply doing what it says). A lone "Done." is judged by what follows it: before another such report, or with nothing after it, it goes; before something the reply wrote, it stays. When the user had asked ELI to do something, the reply then ends "Nothing was carried out in this turn."; under a reply to a greeting or a question the sentence just goes. When the user's own commands did run in the turn (a message of commands and talk), their results are in front of the model and it may refer to them. A fix said to have been made "so it won't happen again" is dropped on any turn.
- **Saying what ELI just did is a question about it.** "then you just stopped spotify", "you just maximised your GUI, not the browser", "what just happened?" are held to the record like "why did you …": the last is answered from the ledger with no model call.
- **A day in a reply has to be a day with records.** For a question about a period, a sentence that puts something on a day with nothing recorded is dropped. "this morning", "earlier today", "yesterday" and "last night" are days too, and the message being answered does not count as a record of its

own day: asked about the past week, ELI had put Thursday's songs down to "this morning" in the day's first conversation.

- **The self-model says so too.** The live self-model line states that logging is always on (every action in the evidence ledger, every turn in the audit chain) and is not something a reply can change.
- **An insult is not a stance.** A complaint, insult or correction aimed at ELI is never recorded as a position ELI holds ("it is 2026 you fucking idiot" was stored as a stance on "2026 idiot").
- **ELI's diagnoses are not the user's facts.** A session summary's "current work" is stored as the user's project only if the user's own words carry it; text they quote back from ELI does not count, and work that is about ELI itself is not stored.
- **Words written on an earlier day keep their day.** Session summaries, recalled memories and earlier sessions' lines reach the prompt dated, and their own "last night", "yesterday", "today" and "right now" become that day's dates (`evidence_format.as_of`, `dated`). A 15 October summary said "zero hours last night"; on 20 October ELI told the user they had mentioned zero sleep last night, and after the correction that they were running on fumes.
- **What the hardware is doing is measured, not said.** In a turn with no evidence, a sentence that puts a state (idle, unused, gathering dust, busy, a percentage, hot) on a part ELI can measure (GPU, card, VRAM, CPU, RAM, the cards' own model numbers) is dropped (`output_governor._hardware_state_claim`): "your RTX 2060 Supers are sitting at idle" was said while a model filled both cards.
- **A promise that did not run says so.** When follow-through decides ELI's own "I am going to check…" is not to be run, the reply ends with a line saying nothing ran for it.

## Behaviours that keep answers grounded

- **Admin and persona actions are explicit only.** The LLM intent resolver cannot choose `SET_USER_NAME`, persona locks, tone and voice changes, memory deletion, plugin or MCP changes, self-patching, shell or training actions (`llm_intent._EXPLICIT_ONLY_ACTIONS`); only a router rule matching the user's words can. A guessed `PERSONA_LOCK_CLEAR` once ran before the turn was downgraded to chat.
- **A name is set only by a declaration.** One extractor (`identity_validation.extract_explicit_identity_facts`) is shared by the router, the knowledge graph, working memory and the executor; hypotheticals and negations ("if my name is listed as unknown…", "my name is not listed") set nothing.
- **Self-patch only patches what it can ground.** `generate_code_patch` extracts the target file from the error traceback; when a failure has no in-project file (e.g. an HTTP/connection error, "No commands to run"), it used to ask the model anyway, which **invented** a path (phantom `api_client.py`) that then failed to apply. It now returns `no_groundable_file` and routes the failure to goal-autogenesis / self-heal instead; when a file IS grounded it is **pinned** in the prompt so the model can't drift to a different path.
- **Banter guard:** the `llm_intent` resolver occasionally maps playful input to a generative action; the engine now downgrades GENERATE_SCRIPT/PROJECT/DOCUMENT/CODE_SOLVE to CHAT when the text carries no create-intent keyword (real "write me a script" is preserved).
- **Compact grounded synthesis carries a VOICE primer** (pulled live from the canonical persona) so factual/introspection answers sound like ELI without losing the EXACT-FACTS contract that pins every number/path/table/DB to the evidence.
- **Rules only where they apply.** The grounding and conversation rules live in `kernel/prompt_rules.py`; `select_rules` puts each guard in the prompt only on a turn it can apply to (self-mechanism rules when the user asks about ELI, the dates rule when dated evidence is present). The whole block used to precede every prompt, 6,800 characters in front of "morning eli", and a small model answered the rules instead of the user. `all_rules()` is the full text, used if selection fails.
- **The clock and the calendar are facts in the prompt.** `evidence_format.time_facts` works out every clock time the user's message names against now ("7.30pm = 19:30 today, 10 h 24 min from now"), and `agenda.prompt_line` lists what is on the calendar for the next 36 hours. A question about either is never escalated to a web search (`grounding_escalation`: `asks_date_arithmetic` or `agenda.about_agenda`).
- **A compound time question is not answered with the clock alone.** "what time is it and how

long until the presentation?" goes to the model with those facts instead of to `TIME`.

- **The profile holds what the user said about themselves.** `profile_extractor. purge_software_talk` runs at start and removes profile rows that are ELI's own troubleshooting stored as facts about the user ("currently frustrated with ELI's memory retrieval"), then rebuilds the user model from what is left.
- **A report about ELI answers a question about ELI.** "help me with my essay" is a request for help, not for the capability list (`HELP` needs a bare "help" or a question about commands). "what is a knowledge graph" and "how does memory work in the brain" are questions about the world, not about ELI's memory (`EXPLAIN_MEMORY_RUNTIME` needs ELI as the subject or an unmistakable reference to its own stores). "how do i improve my memory" is about the user's own. A duration ("in 20 minutes") is not a complaint about speed.
- **A word on its own is not the user's name.** A lone capitalised word set the name ("Dublin", "Tuesday"). It does so only when ELI's last message asked what to call them.
- **A day says what a request is about, not when to do it.** "plan a study session for tonight" and "check the weather for tomorrow" are answered now. A background job needs a clock time, a delay, "overnight", a named report, or a word that asks for it later (`_EXPLICIT_WHEN_RX`, `_DEFER_RX`); with one, "build me a script at 2am" and "run the test suite tonight" are queued instead of run on the spot.
- **Asked about tomorrow, the weather answer is tomorrow's.** `GET_WEATHER` returns the forecast for the day named, not the current conditions; with no place it asks "Where?" once and remembers the answer for the session.

## Deterministic reports are surfaced verbatim; examiner correctness

- **Verbatim guard for deterministic grounded reports.** `_get_chat_response` now returns the EXAMINE_CODE tiered report and the FILE_AUDIT file-count VERBATIM when they appear in the evidence, and surfaces the FIX_FILE success event as a real outcome — instead of letting the quick chat path re-narrate them. This killed three live confabulation classes: EXAMINE_CODE inventing "findings" that were actually existing code comments; FILE_AUDIT inventing files-with-bugs that don't exist; FIX_FILE narrating "I cannot fix" after it had already written the file.
- **Examiner correctness (`code_examiner.py`).** Tier-2 now drops pyflakes star-import noise ("may be undefined, or defined from star imports" — ~800 false positives on the GUI → 0). Tier-3 reviews the WHOLE file in line-correct windows (was only the first ~MAX_FILE_CHARS, so god-files were ~90% unreviewed), each source line prefixed with its REAL number, and rejects any finding citing a line outside the shown window (kills the "engine.py:132 undefined 'ov' " hallucination class — line 132 is `or result.get("error")`).
- **Routing:** "is there any issues with the files?" / "check the code for bugs" now route to EXAMINE_CODE (the real examiner), not FILE_AUDIT; fix-intent ("fix the bugs in foo.py") stays FIX_FILE.

## News topic deepen (grounded, not guessed)

After a `NEWS_FETCH` briefing, a follow-up like **"go deeper into the transformer story"** must not route to open `CHAT` and let the model invent details. The engine (`_try_news_topic_deepen_reroute` in `engine.py`) detects `extract_deepen_topic()` within a short window of the last news command and **re-routes to `NEWS_FETCH` with that topic** — same live fetch path as the original briefing.

The router's general follow-up nudges (`eli.followup.*`, "go deeper", "tell me more") do not count as explicit commands here, so they never outrank this rule: "go deeper on the AI and Caine prize" after a briefing went to chat until 2.5.15.

If the user complains that ELI guessed (`is_news_fetch_complaint`), the kernel recovers the deepen topic from recent conversation (`recover_recent_deepen_topic`) and runs `NEWS_FETCH` again instead of falling through to web-escalation hedge.

---

## Challenging a grounded answer must re-check it, not confabulate a new one

After a grounded action (`RUNTIME_AUDIT`, `RUNTIME_STATUS`, `GPU_STATUS`, `MEMORY_STATUS`, and ~20 other status/diagnostic actions), a detail or challenge follow-up must rebind to the *same* action for a fresh evidence pull, not fall through to open `CHAT` — that lets the model free-associate over whatever it remembers saying, and a skeptical push ("you sure about that?") can make it retract a correct answer and invent a wrong one instead (`_eli_phase19_rebind_grounded_followup` in `engine.py`). Two ways this can silently not fire:

- the follow-up's wording doesn't match `_ELI_PHASE19_CHALLENGE_FOLLOWUP_RX` / `..._DETAIL_...RX` ("and you think the maths checks out??" wasn't covered until it was reproduced live — GPU_STATUS correctly explained requested-vs-effective GPU layers, then confabulated a different, wrong breakdown when challenged in exactly this phrasing);
- the prior action isn't in `_ELI_PHASE19_GROUNDED_FOLLOWUP_ACTIONS` — `GPU_STATUS` and most of its sibling status/diagnostic actions were missing from this set entirely, despite publishing `grounded=True, evidence_used=True` through the identical `_publish_last_response_meta` call site as the actions that were already covered.

`tests/test_phase19_grounded_followup_truth_lock.py` pins both the regex and the action set against the live failure.

**Widened 2026-09-29** for file-backed evidence, not just status/diagnostic actions: a real transcript showed `SUMMARIZE_FILE` correctly reading a document once, then two follow-up complaints ("that's not what I asked" / "you're making this up") producing fabricated analysis — invented citations, fake quoted "answers" — because the rebind didn't cover `FILE_EVIDENCE_ACTIONS` (`SUMMARIZE_FILE`, `READ_FILE`, `ANALYZE_PDF[_FOLDER]`, and 7 more). `_ELI_PHASE19_GROUNDED_FOLLOWUP_ACTIONS` now unions with `FILE_EVIDENCE_ACTIONS` (imported, not hand-duplicated) and the rebind also recovers the prior action's `args` (`path`/`file`/`folder`/`url`) from `_publish_last_response_meta`, so it knows *what to re-examine*, not just *that* to re-examine — re-reading the same file rather than guessing from memory of what it said. The challenge/detail regexes were widened too (`"you did n't really..."`, `"that's not what I asked"`, `"more of a X than..."`, `"you're lying"`, `"making stuff up"`).

## History is not discarded before the budgeter sees it

The old memory-evidence module capped recent-history pulls silently; it was removed and shared turn retrieval (`memory/retrieval.py`) replaced it. The user-facing setting `cog.mem_recent_turns` (default 24, maximum 80) and `cognition/context_budget.py` now decide how much history and memory enter the prompt, and a recall question keeps a floor of a third of the window for memory context.

## Retrieved evidence can't be cut away by an oversized-prompt truncation

Getting evidence into `_build_enhanced_system`'s SITUATION BRIEF / CONVERSATION HISTORY blocks is not enough — if the assembled system prompt then overflows the model's context window, whatever truncates it back down to size has to know where the evidence lives, or it can cut the one thing the whole grounding pipeline exists to protect.

`_get_chat_response`'s hard prompt-size guard used to slice the oversized prompt by raw character position: keep the first half, keep the last half, drop the middle. Its own comment claimed this kept "the grounded-evidence tail (appended last)" — wrong. `_build_enhanced_system` actually writes SITUATION BRIEF and CONVERSATION HISTORY between the persona head and a closing block of generic RESPONSE DISCIPLINE rules, so on a heavily-retrieved turn (a large brief plus a large history) the blind slice could land past the evidence block on both ends, keeping only persona boilerplate and rules with zero retrieved memory inside them. A chain-of-thought turn hit this live: retrieval genuinely found real memory (11 items in the time window, 36 turns, 4575 chars assembled), but the truncated prompt handed to the model contained none of it, and the final answer truthfully — from what it was shown — denied any relevant memory existed.

`_truncate_system_prompt_preserving_evidence` (`engine.py`) fixes this by finding the SITUATION BRIEF / CONVERSATION HISTORY span first and keeping it whole; only the persona/rules head and the closing-rules tail around it get shrunk, and evidence itself is only trimmed (head+tail, within itself) as a last resort. It's wired into both truncation sites in `_get_chat_response` — the broker path and the no-broker direct-GGUF path — which is the only place any of the four algorithmic reasoning modes (chain_of_thought, tree_of_thoughts, constitutional_ai, self_consistency) generate text; `quick` mode's separate streaming path (`_trim_enhanced_system_for_stream`) already trimmed history in place rather than blindly, so it was never exposed to this. See `tests/test_evidence_survives_prompt_truncation.py`.